详细设计文档修改报告

日期：2026-10-17 **改动对象**：`design.md`（正本，766 → **1038 行**）；`详细设计文档（新）.md`（副本，998 → 1075 行，**仍停在 09:52 同步点**） **终态统计**：13 类 / 43 属性 / 63 方法 / 14 条类间关系边（组合 2、关联 7、依赖 5）/ §5 裁决 26 条 **体例**：每条先写「问题」（查出的毛病、出处在哪、会导致什么后果），再写「处理」（实际改法）。定位均按**当前**章节号。尚未闭环的集中在第八节。

一、语义与链路

Q1 同一条消息在全文有三个名字

问题：实现层旧名与图上设计层名长期混用——`TaskRequest` / `TaskInfo`、`TaskResult` / `ActionInfo`、`ExecResult` / `ResultInfo`、`DeviceAction` / `Freq`、`SystemFeature` / `SystemInfo` 各成一套；设频动作有 `apply` 与 `setFreq` 两套写法；`CPU` / `NPU` 对外的 `setFreq(const Freq&)` 与内部物理写节点的方法同名。后果：用例顺序图、类图、正文三处对不上，同一个东西读者无法判断哪个才是权威名。

处理：全文档单层化，一律采用**图上设计层名**，实现层旧名作废。

旧名	新名	备注
`TaskRequest`	`TaskInfo`	推理请求载荷
`TaskResult`	`ActionInfo`	调度结果载荷
`ExecResult`	`ResultInfo`	频率设置结果
`DeviceAction`	`Freq`	设计层动作类型（设备标识 + 目标频率）
`SystemFeature`（作状态用）	`SystemInfo`	状态与 DQN 输入分离
`apply`	`setFreq`	五级链路统一
`CPU` / `NPU` 内部 `setFreq(int)`	`writeFreq(int)`	避免与对外同名方法撞名
`getFeature` / `getSystemState` / `getlastInfo`	`getSystemInfo`	取状态链三跳同名
`schedManager` 入口 `makeDecision`	`taskAsk`	与用例名一致

同时确立「同链路逐级同名」原则：取状态链 `getSystemInfo` 三跳（`systemMonitor` → `systemInterface` → `systemManager`），设频链 `setFreq` 五跳（`decisionExecutor` → `systemMonitor` → `systemInterface` → `systemManager` → `CPU`/`NPU`）。`decisionMaker` 的决策方法仍叫 `makeDecision(TaskInfo, SystemInfo)`，与 `schedManager::taskAsk` 分工不同，不属重名。

Q2 UC3 的调频动作在 Agent 侧没有出口

问题：`decisionExecutor` 拿到 `Decision` 后无处下发——`systemMonitor` 只有读状态的方法，没有任何下发通道；而类间关系里反倒有一条 `decisionExecutor → decisionMaker` 的跨层连线。后果：UC3 顺序图里

`decisionExecutor → systemMonitor → systemInterface → systemManager → CPU/NPU` 这条五级链在类设计中根本不存在，不可实现。

处理：

项	改动
`decisionExecutor`	新增关联属性 `systemMonitor* monitor`（构造注入、不拥有、可为 `nullptr`）
`systemMonitor`	新增 `setFreq(const Freq&) → ResultInfo`，纯转发
`systemInterface`	新增同名 `setFreq`，继续下传
类间关系	删 `decisionExecutor → decisionMaker`，改为 `decisionExecutor — systemMonitor`，边数 14 → 13
原则	补「Agent 侧通往 System 的出口唯一 = `systemMonitor`」，禁止直连 `systemInterface` / `CPU` / `NPU`（后成为 D-12）

Q3 状态缓存挂错了类，与用例的「后台刷新」不符

问题：原设计把缓存放在 `systemMonitor::lastInfo`，并带一个 `collect()` 按需采集。后果有三：① 与 UC2 顺序图（Timer 触发后台刷新、前台纯读）相反；② `systemMonitor` 是门面，不该持状态；③ 采集频率不可控，每次读状态都可能触发一次 sysfs 访问。

处理：

项	改前	改后
缓存归属	`systemMonitor::lastInfo`	`systemManager::systemInfo`
写入	调用时按需采集	`refreshSystemInfo()` 定时覆盖写
读出	`systemMonitor` 内部加工	`askSystemInfo()` 纯读缓存，`systemMonitor` 只转发
新增方法	—	`refreshSystemInfo()`、`setSystemInfo(const SystemInfo&)`
删除	—	`systemMonitor::lastInfo` 属性、`collect()` 方法
顺序图	—	新增《定时采样刷新》（7 条生命线 + `Timer`）

Q4 采样没有驱动者

问题：顺序图里画了 `Timer` 生命线并标「到期 → `refreshSystemInfo`」，但 12 个设计类里没有这个类；`sample_interval_ms` 配置项没有任何承载者；「后台定时刷新」缺节拍来源。后果：UC7 是空中楼阁。

处理：新增 §3.5 `Timer`（取「只判定到期、不建线程」形态，用户裁定）。

- 属性：`intervalMs`（500）/ `lastTick` / `callback` / `armed`
- 方法：`start(intervalMs, cb)` / `stop()` / `poll()`
- `poll()` **先重置 `lastTick` 再回调**，避免回调耗时累加导致到期漂移

- 不创建任何线程：由监控线程在循环里调 `poll()`，与「主函数划分三函数各起一线程」的进程模型一致（后成为 D-07）
- 类间关系新增边 14 `Timer → systemManager`（依赖，回调内容即 `refreshSystemInfo`）

Q5 一次调度下发了一组动作

问题：原 `Decision` 承载动作组、`execDecision` 遍历逐个下发、`ActionInfo` 是数组结构。后果：与项目申请书「每次决策选定**一台**处理器 + **一个**频率」直接矛盾——CPU 与 NPU 是互斥的，不存在「同时下发」或「逐个下发」语义，`ActionInfo` 的大小恒等、严格对位等约定也全部建立在这个错误前提上。

处理：

项	改后
`Decision`	只承载**唯一**一个 `Freq`（目标设备 + 目标频率）
`execDecision`	不遍历动作组，一次下发、一个结果
`ActionInfo`	一元结构（一条动作 + 一个 `ResultInfo`）
§2.3 用例	用 `alt` 片段表达两条互斥分支
原「批大小 / 严格对位 / 不短路」等表述	全部作废

Q6 三类系统状态被画成一棵继承树

问题：原设计让 `SystemInfo` 继承 `DeviceInfo`、`CPUInfo` / `NPUInfo` 再往下继承。后果：系统级状态与设备级状态被强行同构，而两者字段根本不同（系统级聚合两份设备状态 + 特征分量，设备级只描述单设备）；`SystemStateSnapshot` 又被混进状态类型里，概念叠加。

处理：三类并列、**无继承**。

- `SystemInfo`（系统级）：聚合两份设备状态 + 特征分量，`refreshSystemInfo` 在此汇聚
- `CPUInfo` / `NPUInfo`（设备级）：各自只描述单一设备
- `SystemStateSnapshot`：对外应答载荷（`SystemInfo` + 模型名），**不属于**系统状态类型
- 记录为 D-15

二、体例与结构

Q7 12 个类没有一个构造/析构

问题：方法表只有业务方法——对象从哪来、谁负责释放、注入失败怎么办、组合关系由谁兑现，全无说明。`systemManager` 与 `CPU` / `NPU` 是组合关系，却没有任何方法负责创建与销毁。

处理：每类各补一对构造/析构，列为方法表首两行，并各配算法步骤。

项	内容
构造三原则	只做赋值与对象创建、**不做 I/O**（因此不可失败）；监听 / 模型加载 / 读设备节点推迟到 `start()` / `load()` / 首次 Info；注入指针可为 `nullptr` 且**一律不拥有**
析构三原则	只归还本类独占资源、**不 `delete` 注入引用**、释放失败静默不抛异常；**析构不保证被调用**（进程由 systemd 收尸），正常清理走显式 `stop()`，析构仅兜底

项	内容
组合落地	仅 `systemManager` 在构造内创建、析构内删除 `CPU` / `NPU`；其余类平凡析构

注：四类基础设施后改为全静态（见 Q19），该体例最终只适用 9 个类。

Q8 上一轮补构造/析构把方法表打坏了

问题：补行脚本以「方法表首行」为锚做整行替换，把紧随其后的业务方法行一并吞掉。后果：**5 行业务方法整行丢失、5 行前半段被截断、14 处表内空行把表断成两截**——§3 方法表系统性损坏。

处理：

项	数量	明细
补回整行丢失	5	`taskAsk`、`UDSService::start`、`FileService::read`、`ONNXruntime::load`、`decisionMaker::isReady`
补全前半段截断	5	`schedManager` 入口、`execDecision`、`getSystemInfo`、`getCPUInfo`、`getNPUInfo`
删除表内空行	14	断表处

并改掉脚本习惯：补构造/析构**只在目标行之前插入**，不再整体替换方法表首行；每次改完必查「每行 | 数 = 8、表内空行 = 0」。

三、频率与设备

Q9 频率单位三套并存

问题：CPU 侧标 kHz、NPU 侧标 Hz、`curFreq` / `writeFreq` / 四个配置项又标 Hz。后果：同一个物理量在不同节用不同单位，跨类传递必然错 1000 倍，且「读回来要不要乘」无从判断。

处理：

项	改动
单位	设计层全链路统一 **kHz**，共 13 处
参数名	`freqHz` → **`freqKHz`**，共 12 处
换算	收口在**设备边界**：cpufreq 节点原生即 kHz（读入直解、写入直写，**不换算**）；devfreq 节点原生为 Hz（读 ÷1000、写 ×1000）
提示	§3.12 / §3.13 各加一条「频率单位提示」，写明上述差异（D-18 / D-21）

Q10 契约里的「档位」没人能折算成频率

问题：`actions` 写的是「处理器 + 档位号」，而 `Decision` / `Freq` 要的是「处理器 + 目标频率 kHz」。后果：`decisionMaker` 不持任何设备档位表，运行期无从把档位号折算成频率——这条缺口悬置了三轮，导致运行时链路实际是断的。

处理：

- `actions` 改为直接写**绝对频率（kHz）**，取 CPU 可用频率与 NPU 可用频率之并集，一个编号唯一确定一台处理器及其目标频率
- `makeDecision` 第 4 步改为「查表得目标频率，直接构造 `Freq`」
- 动作可选性不由契约保证，改由 `setFreq` 第 1 步夹取兜底（越界夹到最近合法档位并置 `adjusted`）
- 记录为 D-24

Q11 收窄值无落点、governor 无人保证、CPU 节点名用错

问题（三个独立毛病）：

1. `cpu_freq_min/max`、`npu_freq_min/max` 四个配置项在设计层没有任何落点，配置了也不生效。
2. `scaling_setspeed` 在非 `userspace` 策略下会被内核**拒写**，随后回读仍是旧值——造成「写成功、实际没改」的假成功。
3. CPU 侧频率节点名沿用了 devfreq 命名，真实 cpufreq 节点叫 `scaling_available_frequencies` / `scaling_cur_freq` / `scaling_setspeed`，上板必然失效。

处理：

毛病	处理
收窄值	落 `CPU::freqMin/freqMax`、`NPU::freqMin/freqMax`，由 `systemManager` 构造接收后**注入设备、本类不留存**；收窄在 `readFreqTable` **装配期**执行，不另设限制表；收窄后为空则保留原表并返回 `false`
governor	检查落 `CPU/NPU::setFreq` 第二步：每次设频前读策略节点、非 `userspace` 则写入 `userspace`；**不缓存判断结果**（可自愈）；切换失败 → `ok=false` 且**不再写频率**
节点命名	补 D-17「CPU 侧必须用 cpufreq 语义，不可沿用 devfreq 命名」

Q12 `valid` / `stale` 是不产生信息的字段

问题：`DeviceInfo::valid`、`SystemInfo::valid`、`SystemFeature::stale` 三个有效性标志。后果：新架构下缓存只由 `refreshSystemInfo` 定时覆盖写入，这些字段恒为常数（永远「有效」「不陈旧」）；且 DQN 输入只应放物理可观测量，混入采集链元信息会让训练侧无从对齐。

处理：

层	改动
文档	删 `stale` 特征分量；§3.11 属性说明由「最近一次有效」改「最近一次刷新」；新增裁决「有效性约定（不设字段）」= D-16
语义	读到即视为有效（陈旧度 ≤ 一个 `Timer::intervalMs`）；单次读节点失败**零值兜底、照常组装、不中断采集、不回退旧快照**

层	改动
代码	删 `DeviceInfo::valid`、`SystemInfo::valid`、`SystemFeature::stale`；特征定为 6 维（`ucpu + unpu[3] + cpuFreqKHz + npuFreqKHz`）
复验	`code/MiddleWare` 下 `valid` / `stale` 均 0 处

Q13 §2 用例与类设计对不上（5 处）

问题：用例是「乙方视角」写的，类设计改了之后用例没跟上。具体：UC1 步 8 仍写「拆解动作组」（与二选一矛盾）；后置条件没给 `failCount` 递增这一例外；`systemManger` 拼写错误；步 3 / 步 4 用了半角逗号；末尾重复一段与正文同义的 NPU 对称说明。

处理：逐处改正——步 8 改「下发唯一动作（CPU / NPU 二选一，无需拆解）」；后置补「（`failCount` 递增与随之触发的降级判定除外）」；拼写与标点修正；删除重复段。

四、数据设计

Q14 数据设计缺三块：取值接口、装配归属、封装分工

问题：四件互相关联的事都没说清——

1. `ConfigFile` 正文写「提供逐键取值」，方法表只有 `getConfig` / `configValidate` 两行，§4.3.4 又提到 `has()`：**装配层根本取不到键**。
2. 「装配层」全篇出现三次、无任何定义：谁创建谁、什么顺序、失败了怎么办、`ConfigFile` 活多久，全空。
3. `ModelFile` 有 `loadModel()`，与 `ONNXruntime::load` 功能重复，且没有任何调用者。
4. 契约三道校验分别由谁执行、由谁编排，没写。

处理：

缺口	处理
取值接口	补 `getString(section, key, def)` / `getInt(section, key, def)` / `has(section, key)`；对照表 +3 行、方法表 +3 行、共用算法块；默认值一律走参数 `def`，**本类不设内置默认表**（否则出现第二个配置真源）
装配归属	§4.3 新增「装配归属」块：`main()` **不属于** §1 的 13 个设计类、**不进** §3.14 边表；顺序为创建 `ConfigFile` → 校验 → 解析 → 逐键取值 → 拼路径 → 构造注入 → 装载 `Timer`；缺键取默认、**不阻塞启动**；`ConfigFile` 生命周期止于装配期（D-23）
`ModelFile`	删 `loadModel`；写明「不提供读取方法——读取即建会话，由 `ONNXruntime::load` 承担」；三段分工 = `ModelFile` 文件级 / `ONNXruntime` 会话级 / `decisionMaker` 配对级
校验归属	§4.4 新增「执行归属」块：第 1、2 道归 `ContractFile::contractValidate`，第 3 道归 `decisionMaker::loadContract`，统一由 `setModelPath()` 编排（D-25）

Q15 路径参数过多，且目录与文件名可能配不成对

问题：配置里堆了 `model_dir`、`contract_dir`、`model_file`、`contract_file`、`default_model`、`infer_timeout_us` 六个键。后果：目录与文件名分开配 → 可以配出「目录指向不存在的路径」；契约名与模

型名分开配 → 可以配出「换了模型但契约还是旧的」，而这类错误在运行期只表现为「推理结果莫名其妙」。

处理：

项	改动
目录	`model_dir` / `contract_dir` 两个键**删除**，目录固定为 `#define MODEL_DIR "/usr/lib/schedd/models/"`；运维必须把 `模型名.onnx` 与同名 `模型名.json` 放进该目录，其他位置不读
契约名	`contract_file` 键**删除**，契约名由模型名换扩展名派生（`.onnx` → `.json`），基名恒同
模型名	`default_model` 键**删除**：模型名只由 `model_file` 去扩展名派生，不设独立键
超时	`infer_timeout_us` 键**删除**：阈值由 `decisionMaker` 方法内固化，超时按推理失败处理
宏	五个路径全升 `#define`：`CONFIG_PATH` / `MODEL_DIR` / `CPU_FREQ_PATH` / `NPU_FREQ_PATH` / `PROC_STAT_PATH`，公共头文件唯一出处，预处理期字面量替换（不是变量、也不是常量对象）
结果	`[agent]` 节只剩 `model_file` 一个键；配置键总数 6 → 1（模型侧）

记录为 D-22。

Q16 校验严、取值松，两套标准打架

问题：§4.5.2 第 4 步写「整数键须能解析为整数，否则校验失败」，§4.3.4 写「整型项遇非法字面量一律返回默认值」——同一个非法值，一处判失败、一处静默取默认。另外第 4 步还要求「路径键非空」，而 `model_file` 留空是合法的（表示停用模型）。

处理：统一容错——**唯一失败判据是「节 / 键缺失」**；取值一律容错，整数留空或非法取默认、路径留空即退化（模型停用 / 用宏默认），均不判非法。§4.3.4 补「校验与取值同口径」段，明确不存在「校验严格、取值宽松」两套标准。

Q17 模型名、模型路径、契约三个来源

问题：`schedManager::model` 与 `decisionMaker::modelPath` 职责不清；`setModel` 该传名字还是路径不明；`contractPath` 又单占一个属性。更硬的一条：`loadContract` 第 4 步要「比模型输入维度与 `input_dim`」，但此时新会话尚未建立，实际比的是**旧模型**的维度——假校验。

处理：

项	结论
职责	`schedManager::model` 只存**模型名**（初值 `static`）；`decisionMaker::modelPath` 只存**完整路径**——该类不关心「是什么模型」，只关心「模型在哪」

项	结论
入口	`setModel` / `setModelPath` **只收一个完整路径**；模型名由路径末段去扩展名派生（`.../dqn.onnx` → `dqn`）
契约	契约路径**不存属性**（恒等于当前模型路径换扩展名），随模型成对切换
假校验	`loadContract(contractPath, modelInputDim)` 改**双参**：由 `setModelPath` 先建会话、读出 `inDim` 再传入，比的必是**新会话**
不变式	`model != "static"` ⇒ `modelPath` 非空且会话已建；靠「先设路径（含加载）成功、再改 `model`」的顺序保证，任一失败两者均不变

记录为 D-10 / D-11 / D-25。

五、服务接入与基础设施

Q18 `decisionMaker` 没有推理会话

问题：属性只有 `modelPath` / `contract` / `ready`，但 `setModelPath` 要「建新会话、替换旧会话」、析构要「释放会话」、`makeDecision` 要调 `infer` ——**会话没有任何载体**。同时类间关系把 `decisionMaker → ONNXruntime` 标成「依赖」，而依赖关系不产生属性，与上述需求直接冲突。附带：`ONNXruntime` 只有 `load` / `infer`，**没有读输入维度的接口**，而 Q17 需要 `inDim`。

处理：先按「`decisionMaker` 以值成员持有 `ONNXruntime`」拟定（只加 1 行属性 + 改 1 条边）。用户随后改判「基础设施全部静态」，最终落地为：

- `ONNXruntime::session` 改为 `private static`（进程级唯一，惰性建立）
- 新增 `inputDim()`：静态、无参，返回会话输入首维，无效返回 `0`
- 静态会话生命周期：首次 `load` 建立、再次 `load` 以**替换语义**覆盖（旧会话随之回收）、进程退出由静态对象析构回收；**不设 `reset()`**
- `decisionMaker` 析构**不再释放会话**（静态、跨类共享）
- `setModelPath` 建会话后补「调 `ONNXruntime::inputDim()` 读出 `inDim`」

记录为 D-04 / D-05。

Q19 基础设施被当成实例对象用

问题：`UDSService` 被 `serviceInterface` 以**组合属性** `uds` 持有，`handler` 靠构造注入。后果：这个类没有业务状态，实例化只带来「谁创建、谁销毁、构造参数从哪来」三重负担；且四类基础设施（`FileService` / `ONNXruntime` / `UDSService` / `Timer`）混在 12 个业务类里，体例与其余类不一致。

处理：四类基础设施**全部静态化**（用户裁定）。

类	改动
`FileService`	删构造/析构与算法块；`read` / `write` / `exists` → `public static`
`ONNXruntime`	`session` 静态；删构造/析构；`load` / `infer` / `inputDim` 静态
`UDSService`	5 个属性全静态；删构造/析构；`start` 改**三参** `(socketPath, maxConn, handler)`，回调由 `serviceInterface::start()` 带参传入

类	改动
Timer	新建即为静态类（见 Q4）

连带改动：serviceInterface **删 uds 属性**，析构只调 UDSService::stop()；边 1 serviceInterface ◆— UDSService 组合 → **依赖**（组合边只剩 systemManager 持 CPU / NPU 两条）；类数 12 → 13（+Timer）；§3.5~§3.12 顺移为 §3.6~§3.13，类图章 §3.13 → §3.14；「12 类都有构造/析构」旧体例作废，只有非静态的 9 类保留。

Q20 服务接入层缺三件东西

问题：

1. UDSService **没有接受连接的方法**——只有 route（处理单个请求帧），谁去 accept 没写。
2. max_conn 配置项无落点（属性、参数都没有），配了不生效。
3. serviceInterface 没有报文分派入口：谁来解析报文类型、谁来组织应答帧，全空；报文类型还出现 GET_STATE 与 GetState 两种写法。

处理：

项	改动
分派入口	serviceInterface 新增私有 handleFrame(type, payload) → std::string：TaskAsk / GetState / SetModel 三路分派到三个业务方法，失败不主动断连
接受连接	UDSService 新增 acceptLoop()：阻塞 accept → 连接数达 maxConn 立即关闭新连接（不排队）→ 否则入 conns 交 route → 对端断开即移除 → 退出时清空
并发上限	maxConn 双落点：serviceInterface::maxConn（配置入口）→ start() 时传入 UDSService::maxConn；listen(maxConn) 兼作未 accept 队列长度，不另立配置键
报文名	统一为 TaskAsk / GetState / SetModel

Q21 一致性检查查出的 6 条

问题：全文档交叉核对（§2 用例 / §3 类设计 / §3.14 关系表三方对照）查出 6 处自相矛盾。

#	问题	处理
1	systemManager::sampleIntervalMs 与 Timer::intervalMs **重复承载** sample_interval_ms，且前者已无消费者；析构行与析构算法仍写「若采样循环仍在运行则令其停止」——**采样循环已不存在**	删属性；构造签名/前置/后置/算法同步去参数；析构去该步；有效性块改引 Timer::intervalMs；refreshSystemInfo 说明改「由 Timer 到期回调驱动」
2	UDSService **没有 socketPath 属性**，stop() 却要 unlink 套接字文件——静态化后连构造传参都没了，路径彻底取不到	新增静态属性 socketPath：start() 写入、stop() 用后清空
3	UC2 步 4 / 步 8 写 FileService 返回 NPUInfo / CPUInfo，而 FileService::read 返回 std::string	改为「返回节点文本（std::string），由 NPU / CPU 解析组装」

#	问题	处理
4	UC3 步 3.2 / 4.2 写 `FileService` 返回 `ResultInfo`，而 `FileService::write` 返回 `bool`	改为「返回写入是否成功（`bool`），由 `CPU` / `NPU` 组装 `ResultInfo`」
5	UC1 步 14 写 `UDSService` 返回 `ActionInfo`，而 `route` 返回应答帧字符串	改为「返回应答帧（`std::string`，载荷为 `ActionInfo` 的编码）」
6	§3.11 `setFreq` **没有 `nullptr` 分支**，而 UC3 异常流写着「设备未创建 → `unknown device`」	每条分支先验设备已创建：`cpu == nullptr` / `npu == nullptr` → `unknown device`；S4 互斥性补「失败分支下零台被改动」

第 3、4、5 条是同一个毛病：用例表把「服务返回原始数据 + 设备类加工」压成了一步，**返回值归错了类**。改法统一为——服务返回它真实产出的东西（节点文本 / 写入是否成功 / 应答帧），加工后的结构体归 `CPU` / `NPU` / `UDSService`。

六、文档可追溯性

Q22 裁决散落、无编号、无法追溯

问题：26 个 `> **标题**：` 引用块散在 §3 与 §4，标题风格不一（「裁决依据」「执行归属」「边界提示」「路径与宏」混着来），长短差异极大。后果：评审无法按编号逐条追溯；正文里想引用一条裁决只能抄标题字面（如「见 §3.9「单次语义裁决」」），一旦改标题引用即失效。文档整体观感也确实乱。

处理：

- 正文 26 个块统一为 `> **【D-01】标题**：…`，标题逐条重写、与汇总表**逐字一致**
- 新增 `## 5. 设计裁决与说明汇总`：5 列表（编号 / 类型 / 主题 / 结论要点 / 所在节），共 26 行，末行给出分布「裁决 17、约定 3、提示 3、说明 3」
- 首段声明「**正文为权威表述，本表只作索引**；两者不一致以所在章节正文为准」，避免两处说法打架
- 文内交叉引用改为按编号：D-24 内「见 §3.9「单次语义裁决」」→「见 §3.9【D-13】」
- 顺带修掉 §4.5.2 `configValidate` 仍写「取值格式是否合法」的旧口径，与 Q16 的统一容错对齐

新增裁决的维护约定：末尾续号 + 同步 §5 表，不得只改一处。

七、验证

正本当前状态（脚本实测）：

项	结果
行数 / 行尾	1038 行，1038 个 CRLF、0 个裸 LF、0 个裸 CR
类 / 属性 / 方法	13 / 43 / 63
类间关系	14 条 = 组合 2 + 关联 7 + 依赖 5
§3 三张表	每行 `
§5 汇总表	26 行、管道数全 6；与正文编号 26↔26 一一对应，无缺无多
§2 交叉引用	19 处 `类::方法` 引用中，除 `schedManager::failCount`（属性）外全部命中 §3 方法表

项	结果
图片引用	全部有效

副本当前停在 09:52 同步点（1075 行、纯 LF、56 个标题），**未包含**后续静态化、`Timer`、数据设计六项、§5 汇总等改动。副本同步需按章节映射重做一次，并改写正文内 § 引用（`§3.x`→`§7.x`、`§3.14`→`§5`、`§4.x`→`§10.x`）。

八、仍未闭环

以下问题已定位但**未改**，留待裁定：

#	问题	影响
1	`SystemInfo` 声明为**抽象类**，却按值返回 / 按值传参（`askSystemInfo`、`setSystemInfo(const SystemInfo&)`、`taskAsk` 局部变量）	C++ 不可实现，须二选一：改具体类，或全链路改引用/指针
2	并发语义整体缺失：`acceptLoop` 在 accept 循环内串行读帧 → `conns` 恒 ≤ 1、`maxConn` 名存实亡；`stop()` 要「唤醒」但类里无 eventfd / 自管道；`systemInfo` 跨线程无同步；`start()` 自启 `acceptLoop` 与「三线程在主函数启动」冲突	上板即暴露
3	`Handler` 类型无签名定义；`route` 第 3 步写 `handler(GetState)` 单参，而 `handleFrame` 是双参	类型不可实现
4	`taskAsk` 静态模式分支「跳到 S7」，但 S7 要返回「本次唯一动作的 `ActionInfo`」——静态分支无动作产出，返回载荷未定义	降级路径行为不明
5	副本滞后（见第七节）	两份文档口径再度分叉
6	类图仍有两处需在图形工具里改：`Timer` 的箭头方向应为 `Timer → systemManager`；`ServiceInterface → UDSService` 保持虚线（已与文档一致）	图与文档仅剩方向一处不符
7	`architecture.md` §8「三线程 + 监控线程写共享内存、决策线程读」与设计「进程内缓存、无 `/dev/shm`、决策在 `taskAsk` 内同步」口径不一致	架构与设计对不上
8	代码侧待统一：`CPU.h` / `NPU.h` 的 `freqHz` 注释与签名、`SchedTypes.h` 的 `DeviceAction.freqHz`、`UDSService` 仍是实例类（构造注入 handler、无 `maxConn` / `acceptLoop`）	设计与代码分叉

另：装配与进程生命周期（三线程落点、信号、退出顺序）尚未成章。**注意 §5 已被裁决汇总占用，该章须编号为 §6。**